

Taller 2

Casos de Estudio

- 1. Herramienta de Gestión Comercial (Almacén y Facturación)**
- 2. Plataformas de Software Empotrado en Tiempo Real (DEIMOS Space)**

Integrantes

Santiago Botero
Andrés Centanaro
José Jaramillo

Asesor

Andres Sánchez Martin
Arquitectura de Software



Pontificia Universidad Javeriana
Ingeniería de Sistemas
Bogotá
2025

Investigación: Arquitectura_Taller2

Definición y explicación técnica de los componentes usados

- **MVC / Arquitectura por capas:** patrón de diseño arquitectónico en programación que divide una aplicación en tres componentes interconectados pero independientes: el Modelo, que maneja los datos y la lógica de negocio; la Vista, que es responsable de la presentación visual de los datos al usuario; y el Controlador, que actúa como intermediario, recibiendo las entradas del usuario, interactuando con el Modelo y actualizando la Vista. Esta separación mejora la organización, mantenibilidad y escalabilidad del software, facilitando la colaboración en equipos de desarrollo.
- **Vue.js:** framework progresivo para el frontend. Es un framework progresivo para construir interfaces de usuario. A diferencia de otros frameworks monolíticos, Vue está diseñado desde cero para ser utilizado incrementalmente. La librería central está enfocada solo en la capa de visualización, y es fácil de utilizar e integrar con otras librerías o proyectos existentes. Por otro lado, Vue también es perfectamente capaz de impulsar sofisticadas Single-Page Applications cuando se utiliza en combinación con herramientas modernas y librerías de apoyo.
- **Express + TypeScript:** framework minimalista para APIs, potenciado con TypeScript para tipado. Es un framework de aplicaciones web de Node.js que provee un conjunto robusto de características para aplicaciones web y móviles; cuenta con una gran cantidad de métodos de utilidad HTTP y middleware.
- **GraphQL:** Lenguaje de consulta de código abierto para APIs que permite a los clientes solicitar los datos exactos que necesitan, evitando la sobrecarga de datos al obtener toda la información disponible de un servidor en una única llamada de API.
- **MySQL:** Base relacional de código abierto, conocido por su fiabilidad y popularidad en el desarrollo web, siendo una opción preferida para sitios de comercio electrónico, redes sociales y aplicaciones de alto tráfico. Utiliza el SQL (Structured Query Language) para organizar y consultar datos que se almacenan en tablas compuestas por filas y columnas
- **Prisma:** es una herramienta moderna que simplifica la interacción con bases de datos, actuando como un ORM (Object-Relational Mapper) para aplicaciones JavaScript y TypeScript. Su objetivo es que los desarrolladores interactúen con las bases de datos usando un código familiar en lugar de SQL. Se compone de tres herramientas principales: Prisma Client (un generador de consultas seguro y con autocompletado), Prisma Migrate (para definir y migrar esquemas de base de datos) y Prisma Studio (una interfaz gráfica para gestionar los datos).

Relación entre los temas

- Vue (frontend) consume el API expuesta por Express (GraphQL).
- Express (backend) implementa la capa de negocio y resuelve consultas GraphQL; Prisma actúa como capa de acceso a datos hacia MySQL.
- Docker compone los contenedores: frontend (Nginx o dev server), backend (Node), base de datos (MySQL).

Matrices de análisis

Notas sobre la metodología: cada intersección se marca con "X" cuando la característica o principio aplica claramente al componente.

1) SOLID

Principio	MVC/Capas (estructura)	Vue.js	Express+TS	Prisma / BD
S (SRP)	X (separación front/back/DB)	X (componentes)	X (controladores/servicios)	X (modelos separados)
O (Open/Closed)	X (extensible por capas)		X (extensible con middlewares)	X (modelo extendible)
L (Liskov)			X (tipado con TS evita rompimientos)	
I (ISP)	X (interfaces entre capas)		X (APIs específicas)	
D (Dependency Inversion)			X (TS + inyección posible en servicios)	X (abstracción con Prisma)

Justificación concreta: el uso de TypeScript facilita cumplir principios SOLID en el backend; la presencia de prisma permite abstraer acceso a datos.

2) Atributos de Calidad

Atributo	MVC/Capas	Vue.js	Express+TS	Prisma / MySQL
Mantenibilidad	X	X	X	X

Escalabilidad	X	X	X	X
Rendimiento		X	X	X
Seguridad			X	X
Observabilidad			X	

Justificación: TypeScript y separación por carpetas (src/) aumentan mantenibilidad; Prisma y MySQL ofrecen rendimiento y transacciones; GraphQL reduce overfetching en frontend (mejora rendimiento perceptual).

3) Tácticas

Táctica	Vue.js	Express+TS	Prisma / MySQL
Validación entrada		middleware posible	
Caché		posible a nivel resolvers	caching query
Reintentos/Retry		implementable en cliente	
Migraciones controladas			Prisma Migrations

Evidencia: carpeta prisma/ sugiere uso de migraciones (o intención de migraciones), lo que permite tácticas de evolución seguras

4) Patrones

Patrón	Express+TS	Vue.js	Prisma
Repository / DAO	posible con Prisma client		X
Service Layer	controllers + services en src/		

Middleware	X Express		
Component-based UI		X	

Observación: Estructura backend/src y backend/package.json indican una app modular donde es sencillo tener controllers, services y repositorios.

5) Demanda de las tecnologías

Tecnología	Observación en repo	Demanda general
Vue.js	Presente (frontend)	Alta
Express + TS	Presente (backend)	Muy alta
GraphQL	Documentado endpoint	Alta
MySQL	Usado en Docker	Muy alta
Prisma	Carpeta prisma/	Creciente/Alta

Comentario: el combo Vue + Node/TypeScript + MySQL/Prisma es muy buscado en el mercado, especialmente para desarrollos full-stack rápidos y mantenibles.

Resultado de la investigación

A partir de la inspección del repositorio se identificaron y confirmaron los siguientes puntos técnicos y arquitectónicos:

1. Estructura modular: el proyecto separa claramente frontend y backend en carpetas distintas (frontend/, backend/), y el backend tiene la configuración propia de TypeScript (tsconfig.json) y dependencias (package.json), lo que facilita desarrollo y despliegue independiente.
2. API GraphQL: el README indica un endpoint GraphQL en <http://localhost:3001/graphql>, lo que confirma que la comunicación cliente-servidor está basada en consultas/mutaciones GraphQL (en lugar de un API REST puro).
3. Persistencia y herramientas: la presencia de backend/prisma sugiere que Prisma es la herramienta elegida para modelado y acceso a MySQL. El uso de Docker y docker-compose para montar MySQL en puerto 3307 está documentado en el README.
4. Dockerización: el proyecto está preparado para ejecutarse con Docker (README indica carpeta docker y comandos docker-compose up --build), y existen Dockerfiles en backend/.

- Tipado y buenas prácticas: el uso de TypeScript en backend (tsconfig + estructura src/) es una decisión que impulsa mayor seguridad de tipos y facilidad de mantenimiento.

Diagrama de componentes:

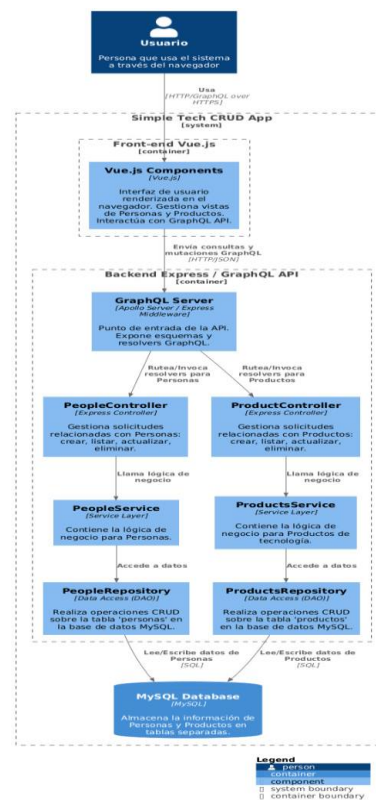
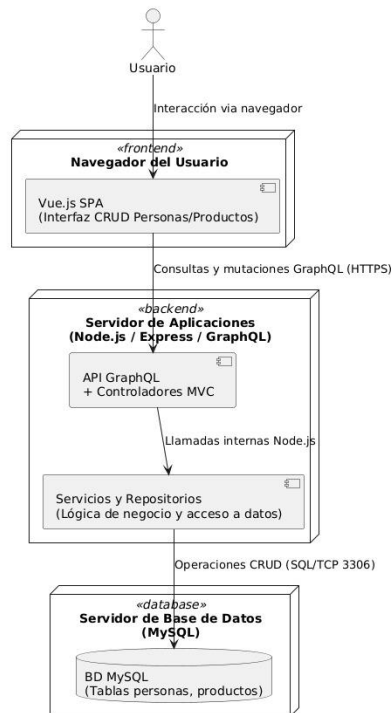


Diagrama de Despliegue:

Diagrama de Despliegue - Simple Tech CRUD App (versión sin librerías)



Conclusión- lo aprendido y aportes del resultado de investigación

Al revisar y trabajar con estas herramientas hemos podido ver de forma práctica cómo se integran tecnologías modernas para construir una aplicación web full-stack. Vue.js demuestra ser una opción muy adecuada para la capa de presentación: su modelo de componentes y reactividad facilita prototipado rápido y mantiene el UI desacoplado de la lógica, lo que ayuda a dividir responsabilidades entre diseñador y backend. En proyectos de tamaño medio, Vue reduce la complejidad del frontend sin imponer demasiada curva de aprendizaje, aunque requiere disciplina en la organización de estados y rutas cuando la app crece.

Express combinado con TypeScript en el backend representa un buen equilibrio entre simplicidad y robustez. Express aporta un flujo de middleware y rutas muy directo, mientras que TypeScript añade seguridad de tipos que mejora la mantenibilidad del código y previene errores comunes en tiempo de compilación. Esta combinación facilita aplicar principios SOLID (por ejemplo, separación de controladores y servicios) y permite refactorizaciones menos riesgosas en fases posteriores del desarrollo.

La elección de GraphQL aporta flexibilidad en la comunicación cliente-servidor: al permitir consultas precisas disminuye el overfetching y simplifica la evolución de la API cuando el frontend necesita datos compuestos. Sin embargo, introducir GraphQL trae complejidad adicional (caching, autorizaciones a nivel de resolvers, monitoreo) y requiere buenas prácticas para evitar que el esquema se convierta en un cuello de botella. En contextos académicos y proyectos donde el frontend consume varias fuentes o necesita optimizar payloads, GraphQL vale la pena; en APIs sencillas REST puede ser sobredimensionado.

Prisma como capa de acceso a la base de datos facilita mucho el modelado, las migraciones y la escritura de consultas con seguridad de tipos. Su integración con MySQL hace que las transacciones y la consistencia sean manejables, y mejora la legibilidad del código frente a consultas SQL crudas. No obstante, hay que ser consciente de las limitaciones cuando se necesitan consultas extremadamente optimizadas o uso intensivo de procedimientos almacenados: en esos casos conviene medir rendimiento y, si hace falta, complementar con queries nativas.

Docker y docker-compose estandarizan el despliegue local y garantizan reproducibilidad entre entornos de desarrollo y pruebas. Esto reduce la fricción entre miembros del equipo y evita "works on my machine". Al mismo tiempo, la dockerización también exige pensar en temas de persistencia de datos (volúmenes), healthchecks y gestión de secrets para no exponer credenciales en entornos públicos. Para pasar a producción, habría que añadir orquestadores/CI, proxies con TLS y políticas de escalado.

Desde la perspectiva de compatibilidad, las tecnologías elegidas encajan bien entre sí: Vue consume GraphQL expuesto por un backend Node/TypeScript que usa Prisma para hablar con MySQL; cada capa es intercambiable si se decide migrar (por ejemplo, reemplazar MySQL por PostgreSQL con cambios mínimos en Prisma). Esa compatibilidad facilita evolución tecnológica y adopción de nuevas herramientas sin rehacer toda la arquitectura.

Como estudiantes de ingeniería, lo que más valoramos es que este proyecto combina buenas prácticas (tipado, separación por capas, dockerización) con herramientas que son relevantes en el mercado. Para mejorar, propondría añadir autenticación y autorización robusta (JWT o similar), pruebas automatizadas (unitarias e integración), métricas/observabilidad y documentación de contrato (GraphQL schema docs o swagger). Estas mejoras no solo endurecerían la solución, sino que también convertirían el repositorio en una referencia más completa para entrevistas o para un portafolio profesional.

En resumen, la arquitectura implementada es coherente y práctica: permite desarrollar, probar y desplegar rápidamente, favorece la mantenibilidad y es altamente compatible con flujos de trabajo modernos. Aprendimos a valorar las decisiones de diseño no solo por su popularidad, sino por cómo facilitan el trabajo en equipo, la evolución del código y la preparación para un entorno productivo.

Referencias

- README principal del repo (instrucciones de Docker, endpoints y puertos)
- frontend/README.md (scripts y comandos Vue)
- backend/ (presencia de Dockerfile, tsconfig.json, package.json, src/, prisma/)
- VUE: <https://es.vuejs.org/v2/guide/>

- GraphQL: <https://graphql.org/#:~:text=%C2%BFQu%C3%A9%20es%20GraphQL?,las%20API%20con%20el%20tiempo>.
- Express: <https://expressjs.com/es/>
- MVC: <https://learn.microsoft.com/es-es/aspnet/mvc/overview/older-versions-1/overview/asp-net-mvc-overview>